

Reviewer Pack — Hamming-Weight Spectrum Slope Bounds ACC⁰[2] Subcircuit Cou...

Entry 1ad89b4c1108 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 19:11:20 UTC

Hamming-Weight Spectrum Slope Bounds ACC⁰[2] Subcircuit Count

Entry ID: 1ad89b4c1108 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 19:11:20 UTC

1. Conjecture

Field A (mathematical branch): Krawtchouk-polynomial spectral analysis: the slope at $k=n/2$ of the weight-distribution polynomial $W_f(z) = \sum_{w=0}^n A_w(f) z^w$ of a Boolean function f , where $A_w(f) = \#\{x \in \{0,1\}^n : |x|=w, f(x)=1\}$; the slope $\sigma(f) = W'_f(1)/W_f(1) - n/2$ is a classical coding-theory invariant (Delsarte/MacWilliams duality) very rarely deployed against circuit lower bounds. **Field B** (complexity object): ACC⁰[2] circuit complexity for Boolean functions presented by truth-table; specifically the minimum number $S_2(f)$ of $\{\text{AND}, \text{OR}, \text{NOT}, \text{MOD}2\}$ -gates in a depth-3 ACC⁰[2] circuit computing f , the Williams/Murray target class for $\text{NEXP} \not\subseteq \text{ACC}^0$ separations.

Statement:

For every Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ with $n \leq 20$, $|\sigma(f)| \cdot \log_2(1 + S_2(f)) \geq \text{Var}_{x \sim U} [|x| \cdot f(x)] / (n+1)$, where Var is taken over the uniform distribution on $\{0,1\}^n$. Equivalently: any ACC⁰[2] circuit of size s computing f must satisfy $s \geq 2^{\{\text{Var}(|x| \cdot f(x)) / ((n+1)|\sigma(f)|)\} - 1}$. A single (f,s) pair with s small but $\text{Var}/((n+1)|\sigma(f)|)$ large refutes the conjecture.

Rationale (proposer's reasoning):

MOD2 gates can erase any single Krawtchouk coefficient, but the slope $\sigma(f)$ is a *signed average* over all weights and is preserved up to a multiplicative log-size factor by depth-3 ACC⁰[2] under random restriction (Smolensky-style approximation by low-degree GF(2) polynomials commutes with weight enumeration). The variance of $|x| \cdot f$ captures the layer-wise informational asymmetry that AND/OR gates can collapse only by multiplying fan-in, giving a Krawtchouk-vs-size trade-off in the spirit of Williams' algorithmic-method size bounds.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b11c06c04fe2728a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all enumerated f for $n \in \{3,4,5\}$ and 2000 random f per $n \in \{6, \dots, 12\}$ (≥ 14000 total instances), compute $r(f) = \text{Var}(|x| \cdot f(x)) / ((n+1) \cdot |\sigma(f)| \cdot \log_2(1 + S_2(f)))$. Conjecture is SUPPORTED iff $\text{support_fraction} \geq 0.99$ AND $\max r(f) \leq 1.0$; FALSIFIED if any (f,s) yields $r(f) > 1.0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.88	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 12 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Krawtchouk weight distribution Boolean function circuit lower bounds - MacWilliams spectrum ACC0 MOD2 circuit complexity - weight enumerator slope depth-3 circuit lower bound Boolean

Top relevant hits considered: - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise) - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/2601.04072v2>] Optimal Monotone Depth-Three Circuit Lower Bounds for Majority

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def binomial(n, k):
    if k > n:
        return 0
    result = 1
    for i in range(k):
        result *= (n - i)
        result //= (i + 1)
    return result

def hamming_weight(x):
    return bin(x).count('1')
```

```

def A_w(f, w):
    n = len(f)
    return sum(f(x) * (-1)**hamming_weight(x) for x in range(2**n) if hamming_weight(x) == w) / bi

def sigma(f):
    n = len(f)
    return sum(A_w(f, w) * math.log2(binomial(n, w)) for w in range(n + 1))

def Var(f):
    n = len(f)
    mean = sum(hamming_weight(x) * f(x) for x in range(2**n)) / (2**n)
    return sum((hamming_weight(x) - mean)**2 * f(x) for x in range(2**n))

def S_2(f):
    n = len(f)
    max_size = 8
    min_size = float('inf')
    for s in range(1, max_size + 1):
        for circuit in generate_circuits(n, s):
            if evaluate_circuit(circuit, f) == f:
                min_size = min(min_size, s)
                break
    if min_size < float('inf'):
        break
    return min_size

def generate_circuits(n, size):
    # Placeholder for circuit generation logic
    # This is a simplified version and may not cover all possible circuits
    if size == 0:
        yield {}
    else:
        for i in range(1 << n):
            for j in range(i + 1, 1 << (n + 1)):
                yield {i: j}

def evaluate_circuit(circuit, f):
    # Placeholder for circuit evaluation logic
    # This is a simplified version and may not cover all possible circuits
    return f

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [3, 4, 5] + list(range(6, 13))
    total_instances = 0
    support_count = 0
    max_violation_ratio = 0
    counterexample = ""

    for n in n_values:
        if n <= 20:
            instances = [random.randint(0, 1) for _ in range(2**n)] * (2000 if n >= 6 else 1)
        else:
            instances = [random.randint(0, 1) for _ in range(2**n)]

```

```

for f in instances:
    A_w_values = [A_w(f, w) for w in range(n + 1)]
    sigma_f = sigma(f)
    Var_value = Var(f)
    S_2_value = S_2(f)

    if sigma_f == 0 or S_2_value == 0:
        continue

    r_f = Var_value / ((n + 1) * abs(sigma_f) * math.log2(1 + S_2_value))
    total_instances += 1
    support_count += int(r_f <= 1)

    if r_f > max_violation_ratio:
        max_violation_ratio = r_f

conjecture_holds = support_count / total_instances >= 0.99 and max_violation_ratio <= 1.0
return {
    "metric_name": "max_violation_ratio",
    "metric_value": max_violation_ratio,
    "instances_tested": total_instances,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample if not conjecture_holds else ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0054871c.py", line 114, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0054871c.py", line 85, in run_trial
    A_w_values = [A_w(f, w) for w in range(n + 1)]
    ^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0054871c.py", line 30, in A_w
    n = len(f)
    ^^^^^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `A_w` (called `len()` on an `int`) before any instances were evaluated, so neither the support nor the falsification criterion can be assessed. | next: Fix the `A_w` helper to accept the truth-table representation actually passed in (or convert `f` to a sequence/array before calling `len`), then rerun the full pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	20607
2	preregistration	claude_max	opus	0	0	6126
3	novelty	claude_max	opus	0	0	3404
4	novelty	claude_max	opus	0	0	8806
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15485
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12721
7	judge	claude_max	opus	0	0	4536

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 71686 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/1ad89b4c1108.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1ad89b4c1108.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1ad89b4c1108.tar.gz (if generated)